

ngspice Linear Solvers — KLU vs. Sparse 1.3

Behavior, defaults, and limitations across DC / AC / transient / noise

Ngspice + OpenVAF Enhancements project

July 2026

Contents

ngspice linear solvers — KLU vs. Sparse 1.3 (behavior, defaults, limitations)	1
TL;DR	1
Selecting and confirming the solver	1
Noise and pole-zero under KLU (Enhancement-113)	2
Two genuine KLU discrepancies	2
The dual-solver test harness	2

ngspice linear solvers — KLU vs. Sparse 1.3 (behavior, defaults, limitations)

This build of ngspice-46 ships two direct linear solvers, and they are not fully interchangeable across analysis types. This note records exactly how they differ — which is the default, how to select and confirm each, and where KLU falls short — based on a direct read of the source and a solver-by-solver sweep of the whole [examples/](#) suite.

TL;DR

	KLU	Sparse 1.3
Availability in this build	Compiled in (SuiteSparse, statically linked — 44 <code>klu_*</code> symbols in the binary)	Always present (ngspice's own solver)
Default?	No	Yes — this build defaults to Sparse 1.3
How to select	<code>.option klu</code>	<code>.option sparse</code> (or just the default)
DC op / DC sweep	✓ correct (one caveat below)	✓ correct
AC	✓ correct	✓ correct
Transient	✓ correct (one caveat below)	✓ correct
Noise (<code>.noise</code>)	✓ correct (since E-113)	✓ correct
Pole-zero (<code>.pz</code>)	✓ single-ended;  balanced-output Sparse-only	✓ correct
AC sensitivity (<code>.sens ... ac</code>)	✗ Sparse-only	✓ correct

Practical guidance: leave the default (Sparse 1.3) unless you have a specific reason to switch. Sparse 1.3 runs every analysis in the suite. Since [Enhancement-113](#) KLU also runs noise and single-ended pole-zero correctly; still Sparse-only under KLU are balanced-output pole-zero and AC sensitivity. Reach for `.option klu` on large, sparse DC/AC problems where KLU's ordering and factorization are faster, and expect it to be less robust on stiff transient edges and degenerate topologies (see the two genuine discrepancies below).

Selecting and confirming the solver

The netlist option is a simple flag:

```
.option klu      * use KLU (SuiteSparse)
.option sparse   * use the legacy Sparse 1.3 (this is also the default)
```

Because unknown `.option` keywords are silently ignored by ngspice, "it ran without complaint" is *not* evidence the solver changed. To confirm which solver is actually active, use the interactive option command, which prints the live CKTKluMODE:

```
.control
run
option      * prints, among other things: "Matrix solver: KLU" or "Sparse 1.3"
.endc
```

On the committed bin/ binary this reports **Sparse 1.3** for a bare deck and for `.option sparse`, and **KLU** only when `.option klu` is given — confirming Sparse 1.3 is the default here.

Noise and pole-zero under KLU (Enhancement-113)

ngspice used to refuse both outright (`noisean.c` / `pzan.c` printed "not (yet) supported with 'option KLU'"). The real reason for noise was subtler than "not wired up": noise uses the adjoint method — it solves the *transposed* system $A^T \cdot x = e$ via `SMPcaSolve` — and `SMPcaSolve`'s KLU branch was calling the non-transposed `klu_z_solve` where its Sparse branch calls `spSolveTransposed`. That silently produced the wrong noise for any asymmetric matrix (every circuit with a transistor or controlled source), which is why it was disabled rather than merely slow.

[Enhancement-113](#) fixes the adjoint solve (`klu_z_solve`) and lifts the guards, so under KLU:

- Noise is correct — verified identical to Sparse on resistor thermal noise, asymmetric VCCS circuits, OSDI device models, and integrated totals. (`.sp` S-parameters share the same adjoint solve and are corrected too.)
- Single-ended pole-zero (grounded output reference) runs correctly.
- Balanced-output pole-zero (non-grounded reference) stays Sparse-only: its zeros phase folds columns at solve time, which Sparse survives via dynamic Markowitz re-ordering but KLU's fixed symbolic factorization cannot; a targeted guard now directs that case to `.option sparse`.
- AC sensitivity (`.sens ... ac`) remains Sparse-only — a separate KLU gap (DC `.sens` works).

Two genuine KLU discrepancies

Beyond the noise/pole-zero refusals, two examples in the suite produce a different (wrong) result under KLU than under Sparse. Both are marked KLU_XFAIL in the [dual-solver harness](#) so they are exercised and tracked rather than silently skipped.

1. `opamp741` — stiff transient diverges. The transistor-level [opamp741 example](#) (a μ A741 from ~70 lines of Verilog-A BJT):

- Sparse 1.3: the large-signal slew test (`pulse(-5 5 ...)` into the follower) runs the full `tran 20n 80u` cleanly (~4058 points), slew rate ≈ 0.54 V/ μ s.
- KLU: the same run diverges at the slewing edge — `v(out)` overshoots past the -5 V rail to ≈ -6.16 V and ngspice aborts the timestep at $t \approx 2.03$ μ s (~133 points), so the characterization can't complete.

A convergence/timestep robustness difference on a stiff circuit: Sparse's pivoting survives the fast edge and KLU's does not. (E-111's line search does not help — it damps the DC-op Newton, not per-timepoint transient iterations.)

2. `groundcontrib` — wrong DC answer on a degenerate topology. The [groundcontrib example](#) drives a single node `p` with a node-to-ground voltage contribution $V(p) <+ 1.5$ through a load resistor to ground. On this exact same deck, changing only the solver:

```
.option sparse -> v(p) = 1.5    (correct)
.option klu     -> v(p) = 0.0    (wrong)
```

This is a genuine KLU correctness miss (not a refusal or a divergence): on this degenerate single-node system KLU returns zero. It is the reason to prefer Sparse for unusual/degenerate one-node topologies. The ~90 other OSDI examples run correctly under KLU, so the issue is specific to this topology, not OSDI generally.

The dual-solver test harness

Every `verify_*.py` in [examples/](#) is run under both solvers automatically. Each script calls `check_both_solvers(__file__)` (right after importing `_setup`); on a normal run that re-executes the script once per solver — injecting `.option sparse` / `.option klu` into every ngspice deck — and prints a combined verdict:

```
=== BOTH-SOLVER RESULT [ceil]: sparse=PASS klu=PASS => OK ===
```

KLU's remaining unsupported analyses (balanced-output pole-zero, AC sensitivity) are auto-detected and reported SKIP; the two KLU_XFAIL examples above are expected-fail under KLU. Env escape hatches: `NGSPICE_SOLVER=klu|sparse` runs once under one solver; `NG_BOTH=0` disables the dual run.

Sweep result (all 101 machine-checkable scripts): 101/101 OK — every example is `sparse=PASS` with `klu` in `{PASS, SKIP, XFAIL}`, i.e. the two solvers agree wherever KLU is applicable. (The `linesearch` example initially *crashed* under KLU, which [Enhancement-112](#) fixed; [Enhancement-113](#) then moved 10 examples from KLU-skipped to KLU-passing by enabling noise and single-ended pole-zero, leaving only analyses skipped under KLU for its AC-sensitivity sub-test. The two XFAIL entries remain `opamp741` and `groundcontrib`.)

Conclusion: for DC, AC, and transient the two solvers agree across the suite, with exactly two KLU exceptions (the stiff `opamp741` transient, and the degenerate `groundcontrib` DC topology), while noise and pole-zero are Sparse-1.3-only by ngspice design. Sparse 1.3, the default, covers everything — which is why it is the default and why the dual-solver harness keys correctness off it.